

What is Hashing? (English)

Hashing is a process that converts any input data (message, file, password) into a **fixed-size output**, called a **hash value** or **digest**.

Example:

Input: "Hello"

Output:

185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764
826381969

1. What are the properties of a Cryptographic Hash Function?

Properties of a Cryptographic Hash Function

A cryptographic hash function must satisfy the following properties:

1. **Compression**

The function takes an input of arbitrary length and produces a fixed-size output (hash value).

2. **Efficiency**

The hash value $h(x)$ can be computed quickly for any given input x .

3. **One-way Property (Pre-image Resistance)**

Given a hash value y , it is computationally infeasible to find an input x such that $h(x) = y$.

4. **Weak Collision Resistance (Second Pre-image Resistance)**

Given an input x and its hash $h(x)$, it is infeasible to find another input $y \neq x$ such that $h(y) = h(x)$.

5. **Strong Collision Resistance**

It is infeasible to find any two different inputs x and y (where $x \neq y$) such that $h(x) = h(y)$.

1. **Compression (সংকোচন)**

যেকোনো দৈর্ঘ্যের input গ্রহণ করে একটি নির্দিষ্ট (fixed-size) দৈর্ঘ্যের output (hash value) তৈরি করে।

2. Efficiency (দক্ষতা)

যেকোনো input x এর জন্য $h(x)$ দ্রুত এবং সহজে গণনা করা যায়।

3. One-way Property (Pre-image Resistance)

যদি কোনো hash value y দেওয়া থাকে, তাহলে এমন input x খুঁজে বের করা প্রায় অসম্ভব যাতে $h(x) = y$ হয়।

4. Weak Collision Resistance (Second Pre-image Resistance)

যদি input x এবং তার hash $h(x)$ জানা থাকে, তাহলে অন্য কোনো $y \neq x$ খুঁজে বের করা কঠিন যাতে $h(y) = h(x)$ হয়।
তুমি কি আরেকটা আলাদা input ($y \neq x$) খুঁজে বের করতে পারবে, যেটার hash একই?

5. Strong Collision Resistance

যেকোনো দুইটি ভিন্ন input x এবং y (যেখানে $x \neq y$) খুঁজে বের করা কঠিন যাতে $h(x) = h(y)$ হয়।

2. State the applications of Hash function.

Applications of Hash Function

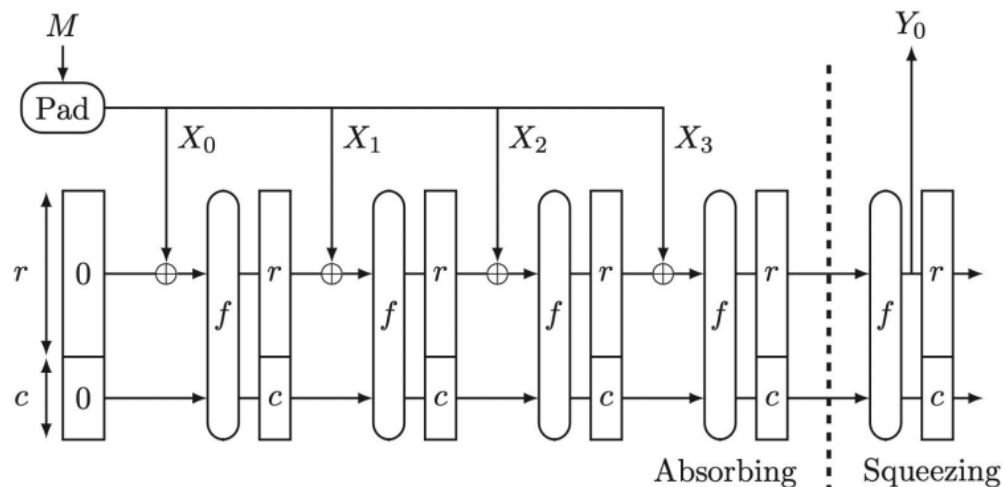
Hash functions are used in many areas of information security. The main applications are:

- **Authentication (HMAC)**
Hash functions are used with a secret key to verify the identity of a user or message.
- **Message Integrity**
Ensures that the message has not been changed during transmission.
- **Message Fingerprinting**
A small hash value acts as a unique “fingerprint” of a large message.
- **Data Corruption Detection**
Used to detect errors or accidental changes in stored or transmitted data.
- **Digital Signature Efficiency**
Instead of signing the whole message, the hash of the message is signed to make the process faster.

- **Symmetric Cryptography Support**
Hash functions are used in different symmetric key based security mechanisms.
- **Other Clever Uses**
Hashing is used in many security protocols and applications beyond basic encryption.
- **• Authentication (HMAC)**
হ্যাশ ফাংশন secret key-এর সাথে ব্যবহার করা হয় যাতে user বা message-এর পরিচয় যাচাই করা যায়।
- **• Message Integrity (বার্তার অখণ্ডতা)**
নিশ্চিত করে যে transmission চলাকালীন message পরিবর্তিত হয়নি।
- **• Message Fingerprinting (বার্তার ফিঙ্গারপ্রিন্টিং)**
একটি বড় message-এর জন্য ছোট একটি hash value তৈরি হয় যা unique “fingerprint” হিসেবে কাজ করে।
- **• Data Corruption Detection (ডাটা নষ্ট হওয়া শনাক্তকরণ)**
সংরক্ষিত বা প্রেরিত ডাটায় কোনো ভুল বা পরিবর্তন হয়েছে কিনা তা শনাক্ত করতে ব্যবহৃত হয়।
- **• Digital Signature Efficiency (ডিজিটাল সিগনেচারের দক্ষতা)**
পুরো message sign না করে শুধু message-এর hash sign করা হয়, ফলে প্রক্রিয়াটি দ্রুত হয়।
- **• Symmetric Cryptography Support (সিমেট্রিক ক্রিপ্টোগ্রাফি সাপোর্ট)**
বিভিন্ন symmetric key ভিত্তিক নিরাপত্তা ব্যবস্থায় হ্যাশ ফাংশন ব্যবহৃত হয়।
- **• Other Clever Uses (অন্যান্য বুদ্ধিদীপ্ত ব্যবহার)**
হ্যাশিং বিভিন্ন security protocol এবং encryption ছাড়া অন্যান্য নিরাপত্তা কাজেও ব্যবহৃত হয়।

3. Draw the block diagram of SHA-3

□ Assuming 4 blocks of input



□ r is "rate", c is "capacity", f is func.

□ $r + c = 1600$, with $c = 256$ or $c = 512$

Lecture-10

1. Evaluate the limitations of traditional password-based authentication in terms of human usability, scalability, and vulnerability to attacks.

Traditional password-based authentication has several important limitations when evaluated from **human usability, scalability, and security (vulnerability to attacks)** perspectives, as highlighted in the dataset.

1. Limitations in Human Usability

Passwords heavily depend on human memory and behavior, which creates major usability issues:

- **Too many passwords to remember**

Users are required to remember multiple passwords across systems, leading to **password reuse**. This is dangerous because if one system is compromised, others become vulnerable.

- **Humans choose weak passwords**
People tend to select predictable passwords like names, dates, or simple patterns (e.g., *frank*, *Password*, *Fido*), making them easy to guess or crack.
- **Hard to manage strong passwords**
Strong passwords (random characters) are difficult to remember, so users often avoid them.
- **User inconvenience**
Frequent password changes and strict complexity rules reduce usability and lead to frustration or unsafe practices (writing passwords down, reuse, etc.).
- **Social engineering risk**
Users can be tricked into revealing passwords, which is a human weakness rather than a technical flaw.

2. Limitations in Scalability (বড় পরিসরে কাজ করার সক্ষমতা)

Password systems do not scale well in large environments:

- **Password management burden**
Administrators must handle resets, recovery, and policy enforcement for many users.
- **High cost of maintenance**
As users grow, support requests (forgotten passwords, resets) increase significantly.
- **Default password issues**
Many systems fail because users do not change default passwords, especially in large deployments.
- **Synchronization problem in large systems**
Managing secure storage, updates, and verification across many systems increases complexity.
- **Password file protection requirement**
Systems must securely store hashed passwords and protect against file compromise.

3. Vulnerability to Attacks

Password-based systems are highly vulnerable, and the dataset highlights multiple attack methods:

(a) Guessing & Dictionary Attacks

- Attackers try common passwords or precomputed lists.
 - Users do not choose random passwords, so attackers have **far fewer possibilities to test**.
 - Dictionary attacks become extremely effective once password hashes are stolen.
-

(b) Hashing and Forward Search Attack

- Passwords are stored as hashes ($h(\text{password})$).
- If attackers obtain the password file, they can:
 - Guess a password
 - Hash it
 - Compare with stored values

(c) Precomputed Dictionary Attack (Rainbow-style idea)

- Attacker precomputes hashes for common passwords.
- If no salt is used, attack becomes extremely fast after initial work.

(d) Weakness without Salt

- Without salt, identical passwords produce identical hashes.
- This allows:
 - Faster cracking
 - Reuse of precomputed dictionary tables

With salt, attack becomes harder but still computationally possible.

(e) Brute Force Attack

- Attackers try all possible combinations.
- Example from dataset:
 - 8-character passwords → very large space, but still feasible with enough computing power.

(f) Human-factor attacks

- Social engineering
- Spyware / keyloggers
- Error logs exposing partial passwords

১. ব্যবহারযোগ্যতা (Human Usability)

পাসওয়ার্ড সিস্টেম মানুষের জন্য কঠিন, কারণ অনেক পাসওয়ার্ড মনে রাখতে হয়। তাই মানুষ একই পাসওয়ার্ড ব্যবহার করে বা সহজ পাসওয়ার্ড (নাম, তারিখ, সাধারণ শব্দ) বেছে নেয়, যা সহজে ভাঙা যায়। শক্ত পাসওয়ার্ড মনে রাখা কঠিন হওয়ায় মানুষ তা লিখে রাখে বা শেয়ার করে। আবার বারবার পাসওয়ার্ড পরিবর্তন করতে হয়, যা বিরক্তিকর ও ঝুঁকিপূর্ণ আচরণ তৈরি করে। এছাড়া social engineering দিয়ে মানুষকে সহজে ফাঁকি দিয়ে পাসওয়ার্ড নেওয়া যায়।

২. স্কেলেবিলিটি (Scalability)

বড় সিস্টেমে পাসওয়ার্ড ম্যানেজ করা কঠিন হয়ে যায়। অনেক ব্যবহারকারীর জন্য admin-দের বারবার password reset ও recovery করতে হয়। ফলে সাপোর্ট কাজ অনেক বেড়ে যায়। বড় ডাটাবেসে পাসওয়ার্ড নিরাপদভাবে সংরক্ষণ ও আপডেট করা জটিল। অনেক সিস্টেমে default password না বদলানোর কারণেও সমস্যা হয়।

৩. আক্রমণের ঝুঁকি (Vulnerability to Attacks)

পাসওয়ার্ড সিস্টেম অনেক ধরনের আক্রমণের শিকার হয়:

(a) Guessing ও Dictionary Attack

হ্যাকাররা সাধারণ বা আগের থেকে জানা পাসওয়ার্ড ব্যবহার করে চেষ্টা করে। যেহেতু মানুষ র‍্যান্ডম পাসওয়ার্ড ব্যবহার করে না, তাই attack সহজ হয়। password hash চুরি হলে dictionary attack খুব কার্যকর হয়।

(b) Hashing ও Forward Search Attack

পাসওয়ার্ড সাধারণত hash আকারে রাখা হয়। যদি attacker password file পেয়ে যায়, সে অনুমান করে password hash করে মিলিয়ে দেখে।

(c) Precomputed Dictionary Attack

আগে থেকেই সাধারণ পাসওয়ার্ডের hash বানিয়ে রাখা হয়। salt না থাকলে এই attack খুব দ্রুত কাজ করে।

(d) Salt না থাকলে দুর্বলতা

একই পাসওয়ার্ড হলে একই hash হয়, তাই attack সহজ হয়। salt দিলে আলাদা hash হয়, তাই attack কঠিন হয়, কিন্তু পুরোপুরি বন্ধ হয় না।

(e) Brute Force Attack

সব সম্ভাব্য কম্বিনেশন চেষ্টা করা হয়। যেমন ৮ অক্ষরের পাসওয়ার্ড হলেও সব চেষ্টা করতে অনেক কম্পিউটিং পাওয়ার লাগে।

(f) Human-factor Attack

Social engineering, keylogger, spyware, এবং error log থেকে পাসওয়ার্ড বের হয়ে যাওয়ার ঝুঁকি থাকে।

2. Assess the use of biometrics for authentication. What trade-offs exist between user convenience, error rates, and security?

✓ English (Easy Answer)

Biometrics (fingerprint, iris, face, etc.) are used for authentication because they are easy for users and hard to copy.

✓ User Convenience

Biometrics are very convenient because users do not need to remember passwords. Just a fingerprint or face scan is enough.

✓ Error Rates

Biometrics are not 100% perfect.

- Sometimes a real user is rejected (insult rate).
 - Sometimes an attacker may be accepted (fraud rate).
- So, there is always some error (called equal error rate).

✓ Security

Biometrics are harder to steal than passwords. But they are not fully secure because:

- Fingerprints or face data can be copied or faked.
- If biometric data is stolen, it cannot be changed like a password.

✓ Trade-off

- More convenience → may increase errors or reduce strict security
 - More strict security → may increase rejection of real users
- So, balancing convenience, accuracy, and security is difficult.

✓ বাংলা (সহজ উত্তর)

বায়োমেট্রিকস (ফিঙ্গারপ্রিন্ট, আইরিস, ফেস রিকগনিশন ইত্যাদি) authentication এর জন্য ব্যবহার করা হয় কারণ এগুলো ব্যবহার করা সহজ এবং নকল করা কঠিন।

✓ ব্যবহারযোগ্যতা (Convenience)

বায়োমেট্রিকস খুব সহজ, কারণ পাসওয়ার্ড মনে রাখতে হয় না। শুধু আঙুল বা মুখ স্ক্যান করলেই হয়।

✓ ত্রুটি হার (Error Rate)

বায়োমেট্রিকস ১০০% নির্ভুল নয়।

- আসল ব্যবহারকারীকে কখনও ভুলভাবে reject করা হতে পারে
 - আবার কখনও আক্রমণকারীকে accept করা হতে পারে
- এটাকে error rate বলা হয়।

✓ নিরাপত্তা (Security)

এগুলো পাসওয়ার্ডের চেয়ে নিরাপদ মনে হলেও পুরোপুরি নিরাপদ নয়, কারণ:

- ফিঙ্গারপ্রিন্ট বা ফেস ডাটা নকল করা যেতে পারে
- একবার ডাটা চুরি হলে তা পরিবর্তন করা যায় না

✓ ট্রেড-অফ (Trade-off)

- বেশি সুবিধা দিলে error বাড়তে পারে

- বেশি নিরাপত্তা দিলে আসল ব্যবহারকারীও reject হতে পারে

☞ তাই biometrics এ সুবিধা, error, আর security এর মধ্যে balance রাখতে হয়।

3. Describe how a salted hash strengthens password security and explain how it mitigates precomputed dictionary attacks.

✓ English (Very Easy Answer)

A **salted hash** means adding a random value (called *salt*) to a password before hashing it.

So instead of:

- `hash(password)`

We store:

- `hash(password + salt)`

✓ How it improves security

- Even if two users have the same password, their hashes will be different because the salt is different.
- Attackers cannot easily guess the original password.

✓ How it stops precomputed dictionary attack

- In a dictionary attack, hackers pre-calculate hashes of common passwords.
- But with salt, each password has a different hash.
- So attackers cannot reuse their precomputed hash table.
- They must compute hashes again for each user → very slow and difficult.

☞ So, salt makes password cracking much harder and prevents precomputed attacks.

✓ বাংলা (খুব সহজ উত্তর)

Salted hash মানে হলো পাসওয়ার্ডের সাথে একটি random value (salt) যোগ করে তারপর hash করা।

অর্থাৎ:

- `hash(password)` এর বদলে
- `hash(password + salt)` রাখা হয়

✓ কিভাবে নিরাপত্তা বাড়ায়

- একই পাসওয়ার্ড হলেও salt আলাদা হওয়ায় hash আলাদা হয়
- তাই attacker সহজে পাসওয়ার্ড বুঝতে পারে না

✓ Precomputed dictionary attack কিভাবে বন্ধ করে

- dictionary attack এ আগে থেকেই অনেক পাসওয়ার্ডের hash বানানো থাকে
- কিন্তু salt থাকলে প্রতিটি user এর hash আলাদা হয়
- তাই আগের তৈরি করা hash list ব্যবহার করা যায় না
- attacker কে আবার নতুন করে hash করতে হয় → অনেক ধীর ও কঠিন

☞ তাই salted hash password security অনেক শক্ত করে দেয়।

4. Discuss the advantages and risks associated with single sign-on (SSO) systems. How do they improve user experience, and what vulnerabilities do they introduce?

✓ Advantages of SSO (Single Sign-On)

Single Sign-On allows a user to log in once and access many systems without entering passwords again.

- **Better user experience:** Users do not need to remember many passwords.
- **Time-saving:** Login only once instead of repeatedly.
- **Less password fatigue:** Reduces frustration from managing multiple passwords.

✓ Risks of SSO

- **Single point of failure:** If one account is hacked, all connected systems become vulnerable.
- **High impact attack:** One stolen password can give access to many services.
- **Difficult recovery:** If SSO account is locked, user loses access everywhere.

☞ Conclusion:

SSO improves convenience but increases risk because one weak point can expose many systems.

✓ SSO-এর সুবিধা

Single Sign-On মানে একবার লগইন করলে অনেক সিস্টেম ব্যবহার করা যায়।

- **ব্যবহার সহজ হয়:** অনেক পাসওয়ার্ড মনে রাখতে হয় না
- **সময় বাঁচে:** বারবার লগইন করতে হয় না
- **ঝামেলা কমে:** password fatigue কমে যায়

✓ SSO-এর ঝুঁকি

- **একটা জায়গা ভাঙলেই সব ভেঙে যায়** (single point of failure)
- **একটা পাসওয়ার্ড চুরি হলে সব access পাওয়া যায়**
- **অ্যাকাউন্ট লক হলে সব জায়গায় সমস্যা হয়**

☞ উপসংহার:

SSO ব্যবহার করা সহজ করে, কিন্তু নিরাপত্তার ঝুঁকি বাড়িয়ে দেয় কারণ এক জায়গার দুর্বলতা পুরো সিস্টেমকে প্রভাবিত করে।

5. In the context of biometrics, define the terms fraud rate and insult rate. In statistics, which of these is a Type I error and which is a Type II error?

✓ Fraud Rate

Fraud rate means **an attacker is wrongly accepted as a real user.**

☞ Example: Trudy is accepted as Alice.

✓ Insult Rate

Insult rate means **a real user is wrongly rejected by the system.**

☞ Example: Alice is not accepted by her own system.

✓ Type I and Type II Error

- **Type I Error = Fraud Rate**
(False accept → wrong person accepted)
- **Type II Error = Insult Rate**
(False reject → real user rejected)

✓ Fraud Rate (প্রতারণা হার)

যখন কোনো আক্রমণকারীকে ভুলভাবে বৈধ ব্যবহারকারী হিসেবে গ্রহণ করা হয়।

☞ যেমন: Trudy-কে Alice হিসেবে গ্রহণ করা।

✓ Insult Rate (অপমান/রিজেক্ট হার)

যখন আসল ব্যবহারকারীকে ভুলভাবে reject করা হয়।

☞ যেমন: Alice নিজেই সিস্টেমে ঢুকতে পারে না।

✓ Type I ও Type II Error

- **Type I Error = Fraud Rate**
(ভুলভাবে অননুমোদিত ব্যক্তিকে accept করা)
- **Type II Error = Insult Rate**
(ভুলভাবে আসল ব্যবহারকারীকে reject করা)

Suppose all passwords on a given system are 8 characters and that each character can be any one of 64 different values. The passwords are hashed (with a salt) and stored in a password file. Now suppose Trudy has a password cracking program that can test 64 passwords per second. Trudy also has a dictionary of 230 common passwords and the probability that any given password is in her dictionary is $1/4$. The password file on this system contains 256 password hashes.

a. How many different passwords are possible?

b. How long, on average, will it take Trudy to crack the administrator's password?

c. What is the probability that at least one of the 256 passwords in the password file is in the dictionary? d. What is the expected work for Trudy to recover any one of the passwords in the password file?

প্রশ্নটা কী বলছে (Easy Explanation in Bangla)

একটা সিস্টেম আছে যেখানে:

◆ **Password system:**

- প্রতিটা password = **8 character**
- প্রতিটা character = **64টা possible value** (A-Z, a-z, 0-9 + কিছু symbol ধরো)
☞ মানে অনেক বড় number of possible password
- Password গুলো **hash + salt করে file-এ রাখা আছে** (মানে direct দেখা যায় না)

◆ **Attacker (Trudy) এর ক্ষমতা:**

- সে প্রতি সেকেন্ডে **64টা password try করতে পারে**
- তার কাছে একটা **dictionary আছে যেখানে 2^{30} (1 billion+) common password আছে**
- probability: যেকোনো password এর **1/4 chance dictionary তে আছে**

◆ **System info:**

- password file এ মোট **256 জন user এর password hash আছে**

✓ এখন প্রশ্নগুলো কী জানতে চাচ্ছে:

(a) কতগুলো possible password আছে?

☞ মানে total password space কত বড়

(b) Administrator password crack করতে গড়ে কত সময় লাগবে?

☞ Trudy brute force / dictionary দিয়ে admin password কত সময় লাগবে (average time)

(c) 256 password এর মধ্যে অন্তত 1টা dictionary তে থাকার probability কত?

☞ মানে পুরো file থেকে অন্তত 1টা weak password আছে কিনা সেটা কত chance

(d) কোনো 1টা password crack করতে expected work কত?

☞ average effort (hash count / time) কত লাগবে যেকোনো user password বের করতে

a. How many different passwords are possible?

✓ Given:

- Password length = 8 characters
- Each character can be chosen from 64 different values

✓ Calculation:

Each position has 64 choices, and there are 8 positions.

So total possible passwords:

$$64^8$$

Now,

$$64 = 2^6$$

So,

$$64^8 = (2^6)^8 = 2^{48}$$

b. How long, on average, will it take Trudy to crack the administrator's password?

Step 1: Average guesses needed

In brute force attack, on average attacker tries half of all passwords:

$$2^{48}/2$$

$$\text{So average tries} = 2^{47}$$

Step 2: Time calculation

Trudy checks 64 passwords per second:

$$2^{47}/64$$

$$\frac{2^{47}}{2^6} = 2^{41} \text{ seconds}$$

✓ Step 3: Convert to years (approx)

$$\begin{aligned} 2^{41} &\approx 2.2 \times 10^{12} \text{ seconds} \\ &\approx 70,000 \text{ years} \end{aligned}$$

- c. What is the probability that at least one of the 256 passwords in the password file is in the dictionary?
- d. What is the expected work for Trudy to recover any one of the passwords in the password file?